Programmation sur Processeur Graphique

—

Partie 7 : Le retour de la gestion de la mémoire (Pinned et Unified Memory)

Pierre-Emmanuel Hladik
pierre-emmanuel.hladik@ec-nantes.fr

25 février 2025

Retour sur les transferts mémoire CPU-GPU

Pour être efficace, l'appel à `cudaMemcpy()` utilise une unité matérielle **DMA (Direct Memory Access)**.

DMA

L'accès direct à la mémoire DMA (Direct Memory Access) est un procédé informatique où des données circulant de, ou vers, un périphérique (port de communication, disque dur, etc.) sont transférées directement par un contrôleur adapté vers la mémoire principale de la machine, sans intervention du microprocesseur si ce n'est pour lancer et conclure le transfert. [wikipedia]

Le DMA utilise les adresses physiques :

- L'adresse est traduite et la présence de la page est vérifiée pour l'ensemble des régions source et destination au début de chaque transfert DMA.
- Aucune traduction d'adresse pour le reste du même transfert DMA, ce qui permet d'obtenir une grande efficacité.

Utilisation d'une mémoire épinglée (pinned)

Le système d'exploitation peut accidentellement supprimer les données lues ou écrites par un DMA et insérer une autre **page virtuelle** [a] au même emplacement physique.

a. ou comment gérer un espace mémoire plus grand que la mémoire vive...

Mémoire épinglée (pinned) [idem Page Locked Memory, Locked Pages, etc.]

La mémoire épinglée est constituée de pages de mémoire virtuelle qui sont marquées —appel système `mlockall()`— de manière à ne pas pouvoir être retirées par pagination

Le DMA utilisé par `cudaMemcpy()` nécessite que toute source ou destination dans la mémoire de l'hôte soit allouée en tant que mémoire épinglée :

- Lors d'un appel à `cudaMemcpy()`, si une source ou une destination dans la mémoire de l'hôte n'est pas allouée en mémoire épinglée, elle doit d'abord être copiée dans une mémoire épinglée (surcharge supplémentaire)
- `cudaMemcpy()` est plus rapide si la source ou la destination de la mémoire hôte est allouée en mémoire épinglée, car aucune copie supplémentaire n'est nécessaire.

Utilisation de la mémoire épinglée dans CUDA

Fonction CUDA pour déclarer un espace de mémoire épinglée

- La fonction `cudaHostAlloc ( void** pHost, size_t size, unsigned int flags )` alloue un espace de **mémoire épinglée pour l'host**, avec :
  - `pHost` : Adresse du pointeur vers la mémoire allouée
  - `size` : Taille de la mémoire allouée en octets
  - `flags` : Option à utiliser `cudaHostAllocDefault`
- La fonction `cudaFreeHost(void* ptr)` libre un espace de mémoire épinglée avec `ptr` le pointeur vers la mémoire à libérer.

- La mémoire allouée est utilisée sur l'*host* de la même manière que celle créée avec `malloc()` et `free()`
- La seule différence est que la mémoire allouée ne peut pas être paginée par le système d'exploitation
- La fonction `cudaMemcpy()` est environ 2 fois plus rapide avec de la mémoire épinglée

La mémoire épinglée est une ressource limitée. Une sur-utilisation peut avoir de graves conséquences. À consommer avec modération.

Exemple de programme sans mémoire épinglée

```
__global__ void AplusB(int *ret, int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

int main() {
    int *h_ret = (int *)malloc(1000 * sizeof(int));

    // Creation de l'espace memoire sur le GPU
    int *d_ret;
    cudaMalloc(&d_ret, 1000 * sizeof(int));

    AplusB<<< 1, 1000 >>>(d_ret, 10, 100);

    // Transfert explicite de la memoire du GPU vers le CPU
    cudaMemcpy(h_ret, d_ret, 1000 * sizeof(int), cudaMemcpyDefault);

    for(int i = 0; i < 1000; i++) {
        printf("%d: A+B = %d\n", i, host_ret[i]);
    }

    free(host_ret);
    cudaFree(ret);
    return 0;
}
```

Exemple de programme avec mémoire épinglée

```
__global__ void AplusB(int *ret, int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

int main() {

    // Reservation d'un espace epingle sur le CPU
    int *h_ret;
    cudaHostAlloc((void **) &h_ret, 1000 * sizeof(int), cudaHostAllocDefault);

    // Creation de l'espace memoire sur le GPU
    int *d_ret;
    cudaMalloc(&d_ret, 1000 * sizeof(int));

    AplusB<<< 1, 1000 >>>(d_ret, 10, 100);

    // Transfert explicite de la memoire du GPU vers le CPU
    cudaMemcpy(h_ret, d_ret, 1000 * sizeof(int), cudaMemcpyDefault);

    for(int i = 0; i < 1000; i++) {
        printf("%d: A+B = %d\n", i, host_ret[i]);
    }

    cudaFreeHost(host_ret);     // Liberation de la memoire epinglee
    cudaFree(ret);
    return 0;
}
```

Unified Memory

- La mémoire unifiée a été introduite pour la première fois dans CUDA 6.0
- C'est un espace mémoire commun qui est vu par l'*host* et les *devices* comme une mémoire unique avec un espace d'adressage commun
- Le matériel/logiciel gère la migration des données automatiquement entre l'*host* et le *device* en maintenant la cohérence entre eux sans effectuer des appels explicites de copie de mémoire (i.e. élimine l'appel à la routine `cudaMemcpy`)
- La mémoire unifiée permet de :
  - simplifier la programmation GPU en unifiant les espaces mémoire de manière cohérente
  - maximiser la vitesse d'accès aux données par la migration transparente des données vers le processeur qui les utilise

Exigence système

La mémoire unifiée est disponible pour :

- un GPU avec une architecture 3.0 ou supérieure (classe Kepler ou plus récente)
- une application hôte 64 bits et un système d'exploitation adéquat (Linux ou Windows).

Routines d'utilisation

- Il n'est plus nécessaire de procéder à des transferts de mémoire explicites entre l'hôte et le périphérique. Toute allocation créée dans l'espace mémoire est automatiquement migrée vers l'endroit où elle est nécessaire.
- Un programme alloue la mémoire de deux manières :
  - via la routine `cudaMallocManaged()`, qui est sémantiquement similaire à `cudaMalloc()` ; ou
  - en définissant une variable globale `__managed__`, qui est sémantiquement similaire à une variable `__device__`

Exemple de programme sans mémoire unifiée

```
__global__ void AplusB(int *ret, int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

int main() {
    int *h_ret = (int *)malloc(1000 * sizeof(int));

    // Creation de l'espace memoire sur le GPU
    int *d_ret;
    cudaMalloc(&d_ret, 1000 * sizeof(int));

    AplusB<<< 1, 1000 >>>(d_ret, 10, 100);

    // Transfert explicite de la memoire du GPU vers le CPU
    cudaMemcpy(h_ret, d_ret, 1000 * sizeof(int), cudaMemcpyDefault);

    for(int i = 0; i < 1000; i++)
        printf("%d: A+B = %d\n", i, host_ret[i]);
    free(host_ret);
    cudaFree(ret);
    return 0;
}
```

Exemple d'utilisation de cudaMallocManaged

```
__global__ void AplusB(int *ret, int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

int main() {
    int *ret; // adresse commune CPU ET GPU

    cudaMallocManaged(&ret, 1000 * sizeof(int));

    AplusB<<< 1, 1000 >>>(ret, 10, 100);

    cudaDeviceSynchronize();

    for(int i = 0; i < 1000; i++)
        printf("%d: A+B = %d\n", i, ret[i]);
    cudaFree(ret);
    return 0;
}
```

- La routine `cudaMallocManaged()` renvoie un pointeur valable à la fois pour le code de *host* et celui du *device* . Cela permet d'utiliser `ret` sans une copie séparée de `host_ret` ce qui simplifie et réduit la taille du programme
- Dans l'exemple non unifié, la routine `cudaMemcpy()` est utilisée à la fois pour synchroniser le kernel (c'est-à-dire pour attendre qu'il finisse de fonctionner), et pour transférer les données vers l'hôte. L'utilisation de la mémoire unifiée nécessite donc un `cudaDeviceSynchronize()`explicite avant que le programme hôte puisse utiliser en toute sécurité la sortie du GPU

Exemple d'utilisation de __managed__

```
__device__ __managed__ int ret[1000]; // adresse globale commune CPU ET GPU

__global__ void AplusB(int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

int main() {
    AplusB<<< 1, 1000 >>>(10, 100);

    cudaDeviceSynchronize();

    for(int i = 0; i < 1000; i++)
        printf("%d: A+B = %d\n", i, ret[i]);
    return 0;
}
```

Simplifie davantage un programme lorsque des variables globales sont utilisées.

Absence de l'appel à `cudaMemcpy()` et le tableau de retour `ret` est visible à la fois sur le CPU et le GPU.

Migration et cohérence des données

- La mémoire unifiée tente d'optimiser les performances de la mémoire en déplaçant les données vers la mémoire *host* si le CPU y accède et vers la mémoire du *device* si le GPU y accède
- La migration des données est transparente pour un programme. Le système essaie de placer les données à l'endroit où elles sont le plus efficacement accessibles sans violer la cohérence
- L'emplacement physique des données est invisible pour un programme et peut être modifié à tout moment, mais les accès à l'adresse virtuelle des données resteront valides et cohérents à partir de n'importe quel processeur, indépendamment de la localité. **La cohérence est l'exigence principale, avant les performances**
- Les architectures de GPU dont la capacité de calcul est inférieure à 6.x ne prennent pas en charge le déplacement fin des données gérées vers le GPU à la demande. Chaque fois qu'un kernel GPU est lancé, toute la mémoire gérée doit généralement être transférée vers la mémoire du GPU pour éviter les erreurs d'accès à la mémoire

Un peu de pratique avec le TD 7

Objectifs

- Déclaration d'un espace épinglé
- Utilisation de la mémoire unifiée
- Comparaison des performances

Ouverture (très rapide) à la gestion de la mémoire

Pour ceux qui souhaitent aller plus loin...

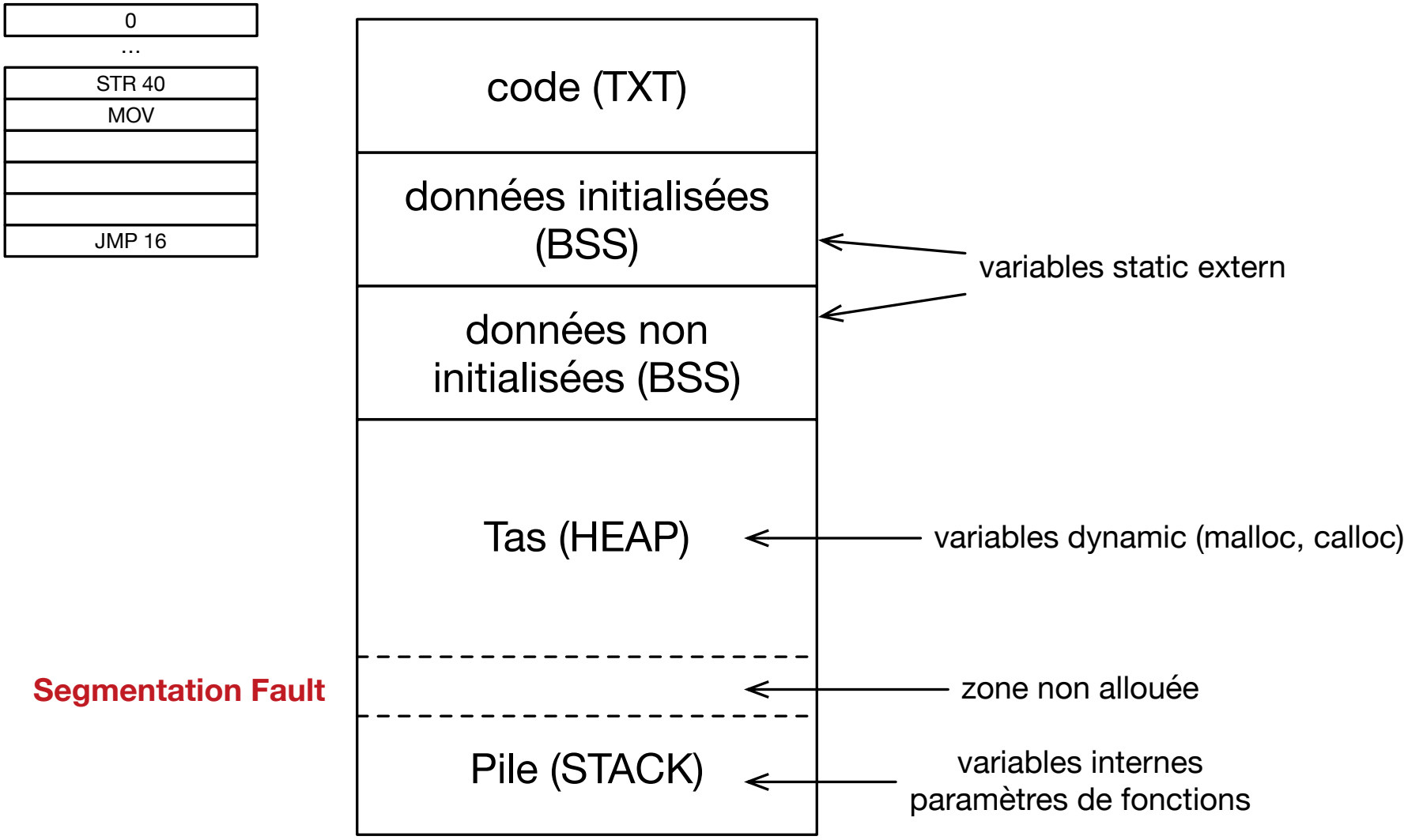
Hiérarchie mémoire

- mégaoctet de mémoire cache volatile : rapide et chère
- gigaoctet de mémoire volatile de rapidité d'accès et prix moyen
- teraoctet de mémoire de masse non volatile lente et peu chère

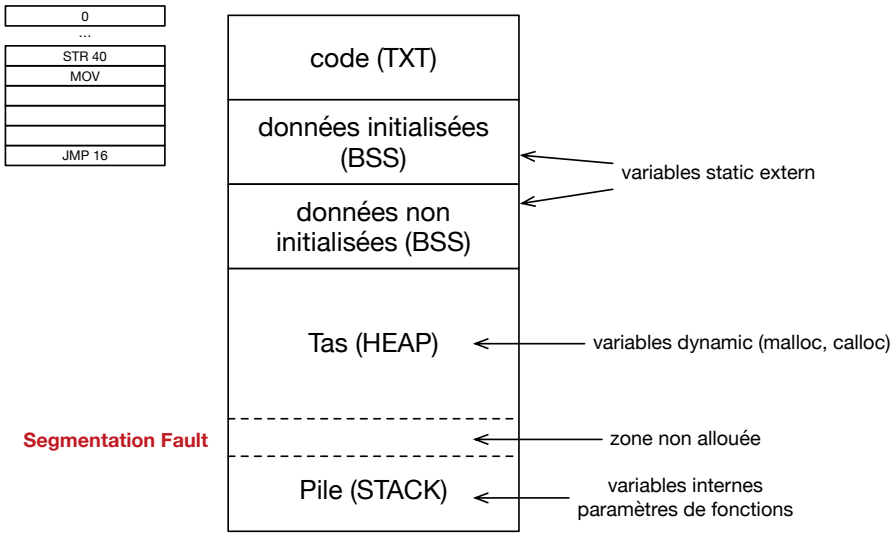
Gestionnaire de mémoire

Un gestionnaire de mémoire est l'entité d'un système d'exploitation qui gère la hiérarchie mémoire et la coordination entre elles.

Structure de la mémoire associée à un programme

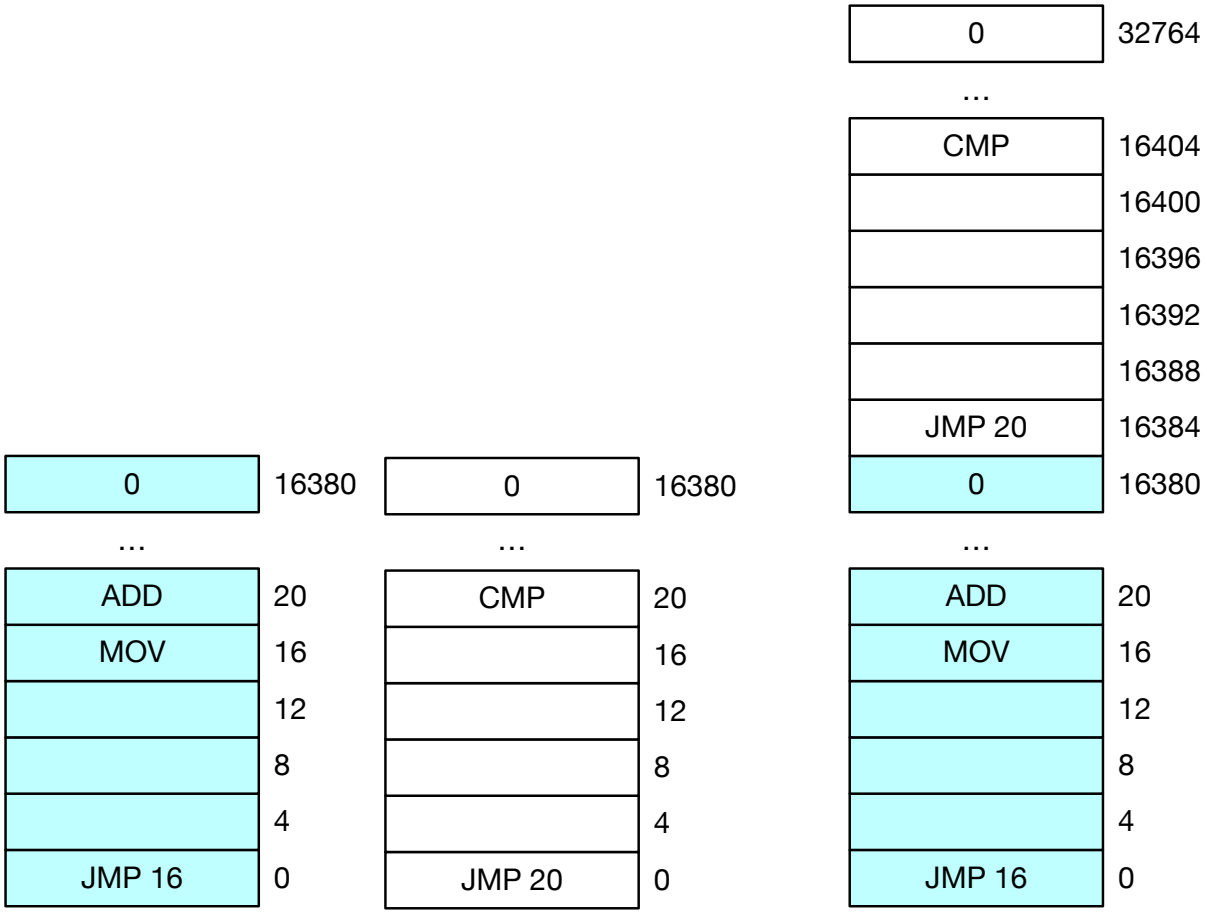


Exemple : Structure de la mémoire associée à un programme

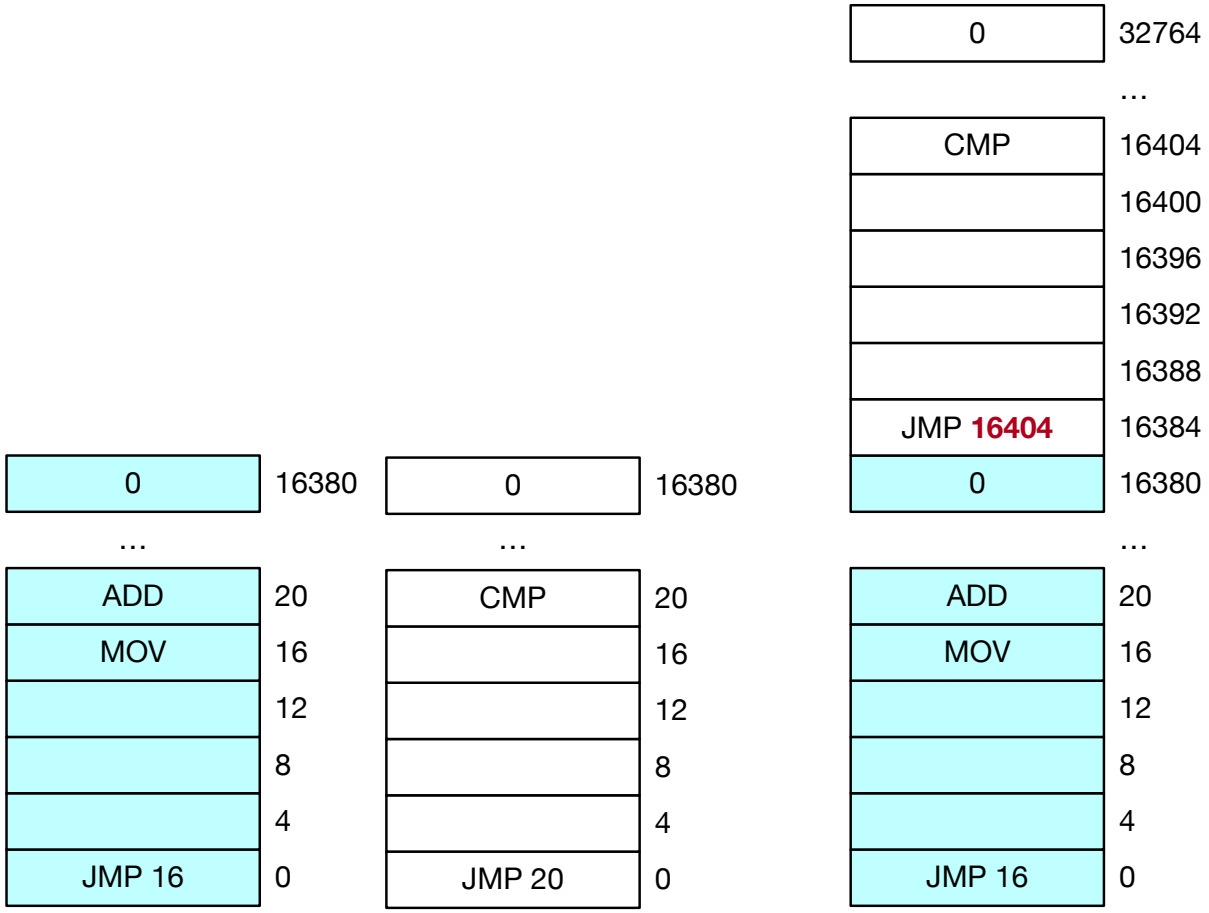



```c
const int a  = 42;
int b[] = { 1, 2, 3 };
int c;

int f(int x, int y)
{
    int *z;
    malloc(z, sizeof(int));
    *z = x;
    *z = *z * y;
    return *z;
}

int main(int argc, char **argv)
{
    const int d = 1664;

    c = f(a, d);

    return 0;
}
```

Et avec plusieurs programmes...



Réallocation statique



Méthode utilisée pour les systèmes embarqués

Espace d'adressage

Définition

Un espace d'adressage est une plage d'adresses valides en mémoire disponibles pour un programme ou un processus, c'est-à-dire que c'est la mémoire à laquelle un programme ou un processus peut accéder.

Chaque programme aura son propre espace, ainsi l'adresse 42 d'un programme correspond à un emplacement mémoire différent de l'adresse 42 d'un autre.

L'espace d'adressage est une sorte d'abstraction de la mémoire utilisée par un programme.

Réallocation dynamique

Définition

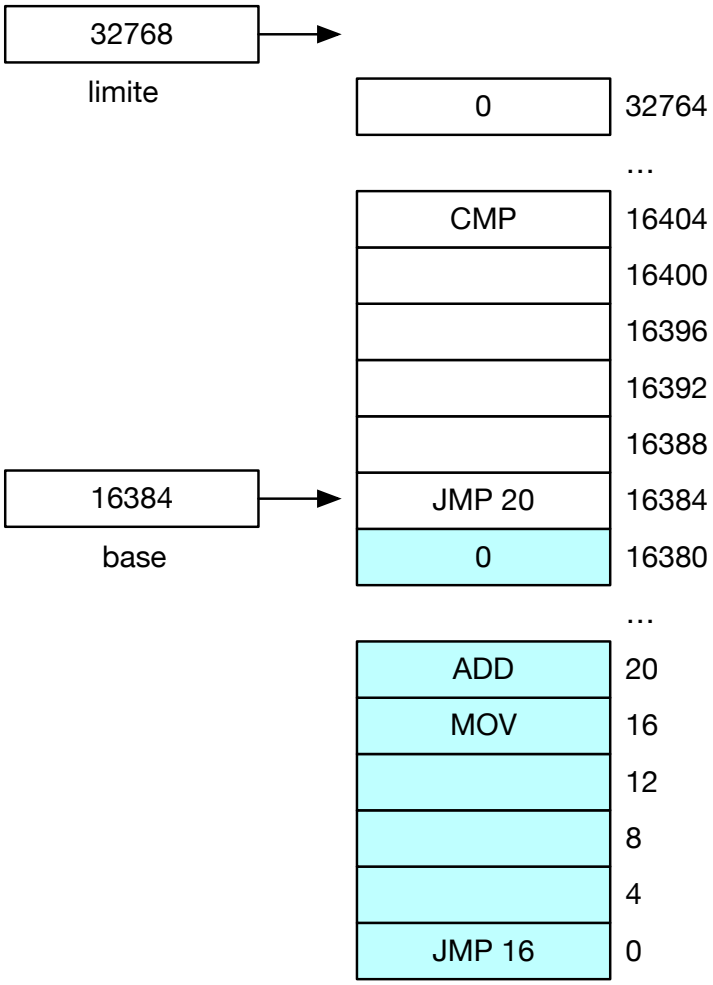
Ajout de deux registres : registre de base et registre de limite.

A l'exécution le système d'exploitation charge dans ces registres l'adresse physique de début et de fin du programme.

On ajoute à toutes les références mémoires l'adresse de base (et vérifie que l'adresse est inférieure à l'adresse limite).

Défaut

Nécessite une addition et une comparaison (comparaison rapide mais addition lente – à cause de la retenue –)



Taille de la mémoire

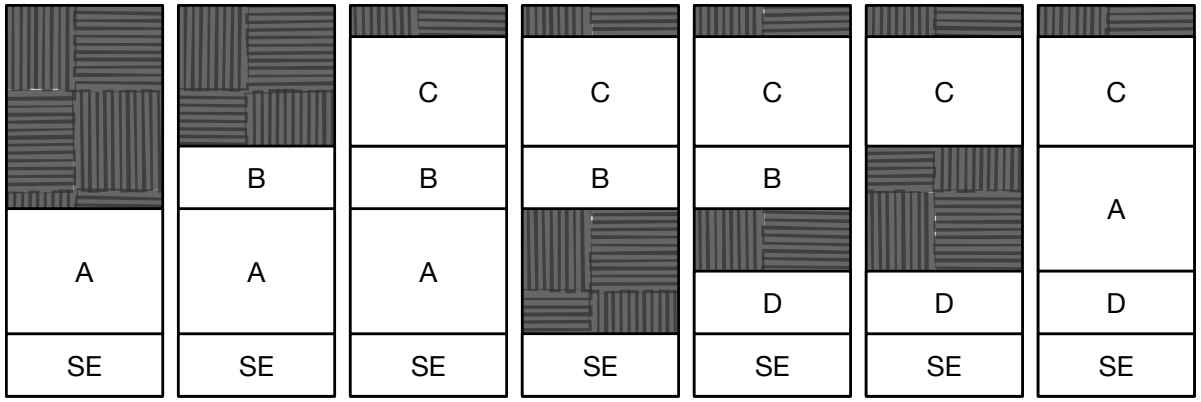
Problème

Comment faire si la mémoire physique n'est pas assez grande pour contenir tous les processus ?

Solution

- va-et-vient (swapping)
- mémoire virtuelle

Swapping



- réservation d'un espace supplémentaire (tas et pile)
- technique de compactage de la mémoire (peu utilisé)
- gestion de la mémoire libre : structures (table, listes, etc.) et algorithmes (first fit, next fit, etc.)

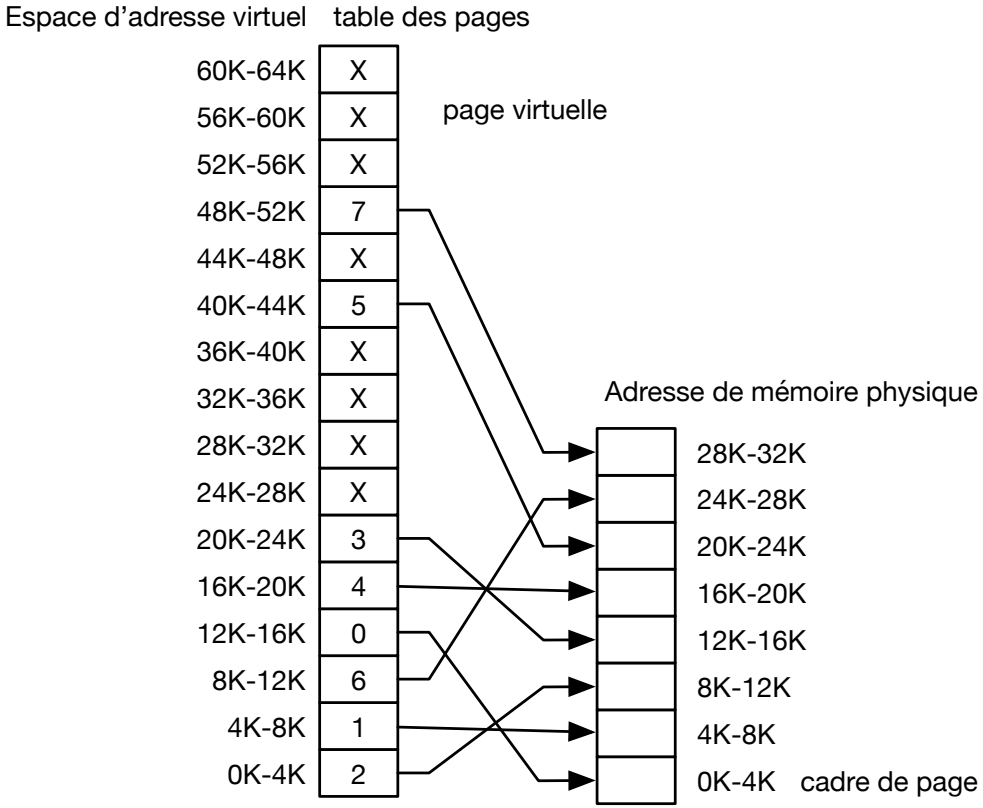
Mémoire virtuelle

Principe

- chaque programme a son espace d'adressage découpé en petites entités, les pages
- les pages sont mappées sur la mémoire physique, mais pas forcément toutes les pages pour exécuter le programme
- la correspondance entre les adressages virtuels et physiques est fait à la volée par le matériel (Memory Management Unit, MMU)
- si une page n'est pas référencée, le système d'exploitation charge l'espace manquant

Pagination

- L'espace d'adressage virtuel est divisé en **pages**
- Les unités correspondantes dans la mémoire physique sont appelées **cadres de pages** (page frame)
- Les pages et les cadres sont de même taille (4ko dans l'exemple)
- Les transfert entre la RAM et le disque se font toujours par pages entières



Défaut de page

Algorithme de remplacement de page

Une instruction fait un accès à une page qui n'est pas présente

1. sélection d'un cadre de page peu utilisé
2. écriture sur le disque du cadre sélectionné
3. transfert de la nouvelle page référencée dans le cadre libéré
4. modification de la table des pages
5. poursuite de l'instruction

TLB (mémoire cache)

Problème

La correspondance de pages doit être rapide et est une contrainte de performance

Observation

Un programme fait des accès à un petit nombre de pages

Utilisation

Mémoire locale à la MMU avec une petite table de correspondance (Translation Lookaside Buffer, TLB).
Le TLB mémorise les derniers couples (page, cadre) correspondant aux dernières pages auxquelles le processeur a dû accéder, ce qui permet d'améliorer grandement les temps d'accès à la mémoire

Et encore

- Page des grandes mémoire
- Algorithme de remplacement de pages (non récemment utilisée, premier entré-premier sorti , LRU, etc.)
- Conception des systèmes de pagination (local/global, taille, séparation instructions et données, etc.)
- Pages partagées
- Bibliothèques partagées
- Segmentation